

Project 2: Formal Criteria-Based Test Design & Automation

System: Premium Discount Eligibility System

1. Input Space Partitioning (ISP) Design

The input space is partitioned into disjoint and complete blocks based on the functional requirements of the DiscountCalculator.

Characteristic	Definition	Blocks
C1: customerType	The category of the customer.	b1: NEW, b2: REGULAR, b3: PREMIUM
C2: totalOrdersInLastYear	Number of orders in the previous year.	b1: [0, 10), b2: [10, ∞)
C3: isSubscribedToNewsletter	Subscription status for the newsletter.	b1: true, b2: false

2. Pair-Wise Coverage (PWC) Test Cases

The following combinations were derived to satisfy Pair-Wise Coverage. These ensure that every possible pair of values between any two characteristics is covered at least once.

Test Case	customerType	totalOrders	isSubscribed	Expected Result / Logic
1	NEW	5	true	7% (5 base + 2 sub)
2	NEW	3	false	5% (5 base)
3	REGULAR	5	true	10% (5 base + 3 type + 2 sub)
4	REGULAR	12	false	

Test Case	customerType	totalOrders	isSubscribed	Expected Result / Logic
				13% (5 base + 3 type + 5 loyalty)
5	PREMIUM	10	true	15% (Capped: 5+5+5+2=17 → 15)
6	PREMIUM	3	false	10% (5 base + 5 type)

3. Infeasibility Justification

Infeasible Combination: `customerType = "NEW"` AND `totalOrdersInLastYear ≥ 10`.

Justification: According to the functional requirements (Constraint #5), a new customer cannot have 10 or more orders from the previous year. Our implementation handles this by throwing an `IllegalArgumentException`, as verified in the `DiscountCalculatorTest.java` (Section 2).

4. Limitations of Pair-Wise Testing

While Pair-Wise Coverage (PWC) is an industry-standard technique for reducing the number of test cases while maintaining high fault-detection rates, it has inherent limitations:

- **Multi-way Interactions:** PWC only guarantees detection of faults triggered by the interaction of two parameters. It does not guarantee the detection of "3-way interaction faults" (or higher) where a bug only occurs when three specific variables align in a certain way.
- **Logic Errors:** Combinatorial testing focuses on input combinations but may not catch logical errors within specific branches of the code if those branches aren't uniquely targeted by a pair.
- **Exhaustive Gap:** Achieving 100% PWC is not the same as 100% exhaustive testing. For a system with many inputs, PWC might only cover a small fraction of the total state space.

5. Team Contribution Matrix

Member Name	Student ID	Primary Contributions
Omar Rami Mahmoud Abu Yousef	2232499	ISP Modeling, Infeasibility Analysis, and GitHub Setup.
Abdulrahman Mustafa Al-Nobani	2044959	JUnit 5 Implementation (Parameterized Tests) and Logic Verification.
Haitham Oudeh Fahad Al-Naimat	2137623	Technical Report Documentation and Limitations Analysis.

GitHub Repository: <https://github.com/Ozr1978/discount-calculator-project.git>